1. Introduction & Data Source

Introduction

This practical investigates the application of machine learning classification algorithms to the Car Evaluation dataset. Two supervised learning models, K-Nearest Neighbors (KNN) and Logistic Regression, were implemented and evaluated using the same training and testing data. The models were compared using accuracy and ROC-AUC to determine which approach performed better on the dataset.

Data Source

The dataset used in this practical is the **Car Evaluation Dataset**, obtained from the UCI Machine Learning Repository. The dataset contains **1,728 instances and 7 attributes** describing car purchasing evaluations. The input attributes include buying price, maintenance cost, number of doors, passenger capacity, luggage boot size, and safety. The target variable is the car evaluation class.

The target classes are:

- unacc — unacceptable
- acc — acceptable
- good — good
- vgood — very good

Dataset URL:
https://archive.ics.uci.edu/dataset/19/car+evaluation

Raw Dataset URL:
https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data

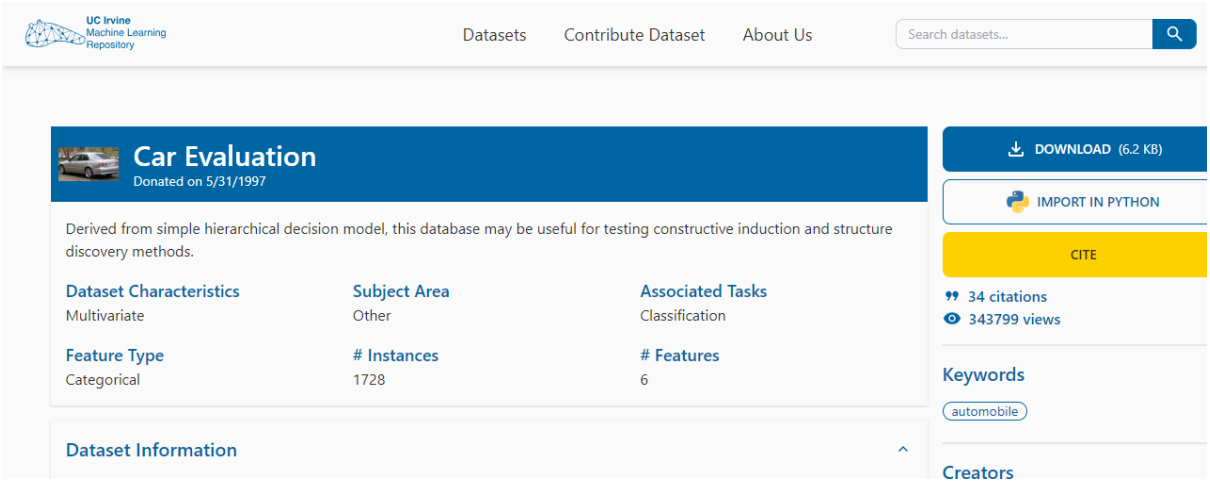


Figure 1: UCI Car Evaluation Dataset Source

2. Data Pre-processing

The dataset was loaded into a pandas DataFrame using the UCI raw dataset URL. The dataset contains 1,728 records and 7 columns. The columns represent buying price, maintenance cost, number of doors, passenger capacity, luggage boot size, safety, and the car evaluation class.

First, the dataset was inspected using **df.head()** and **df.info()** to understand its structure and data types. Missing values were then checked using **df.isnull().sum().** No missing values were found in the dataset.

Since the target variable, class, is categorical, it was converted into numerical labels using LabelEncoder. The input categorical features were converted into numerical form using one-hot encoding. This created 21 numerical input features.

The dataset was then divided into training and testing sets using an 80:20 split with random_state=42. This resulted in 1,382 training records and 346 testing records.

Finally, StandardScaler was applied to the input features. The scaler was fitted only on the training data and then used to transform both the training and testing data. Scaling was performed to ensure that the numerical feature representations were on comparable scales, which is particularly important for distance-based algorithms such as KNN.

Pre-processing summary

Pre-processing step	Result
Original records	1,728
Original attributes	7
Missing values	None
Target encoding	Label Encoding
Feature encoding	One-Hot Encoding
Encoded features	21
Training set	1,382
Testing set	346
Test size	20%
Feature scaling	StandardScaler
Random state	42

```
df.head()
```

0.0s

Python

	buying	maint	doors	persons	lug_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

```
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()
```

3.4s

Python

```
Shape: (1728, 7)

Columns:
['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class']

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1728 entries, 0 to 1727
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   buying    1728 non-null   object
 1   maint     1728 non-null   object
 2   doors     1728 non-null   object
 3   persons   1728 non-null   object
 4   lug_boot  1728 non-null   object
 5   safety    1728 non-null   object
 6   class     1728 non-null   object
dtypes: object(7)
memory usage: 94.6+ KB
```

- **Figure 2:** Dataset loading and inspection (df.head() / df.info())

```
df.isnull().sum()
```

0.0s

Python

```
buying      0
maint       0
doors       0
persons     0
lug_boot    0
safety      0
class       0
dtype: int64
```

- **Figure 3:** Missing-value check

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

df["class_encoded"] = label_encoder.fit_transform(df["class"])

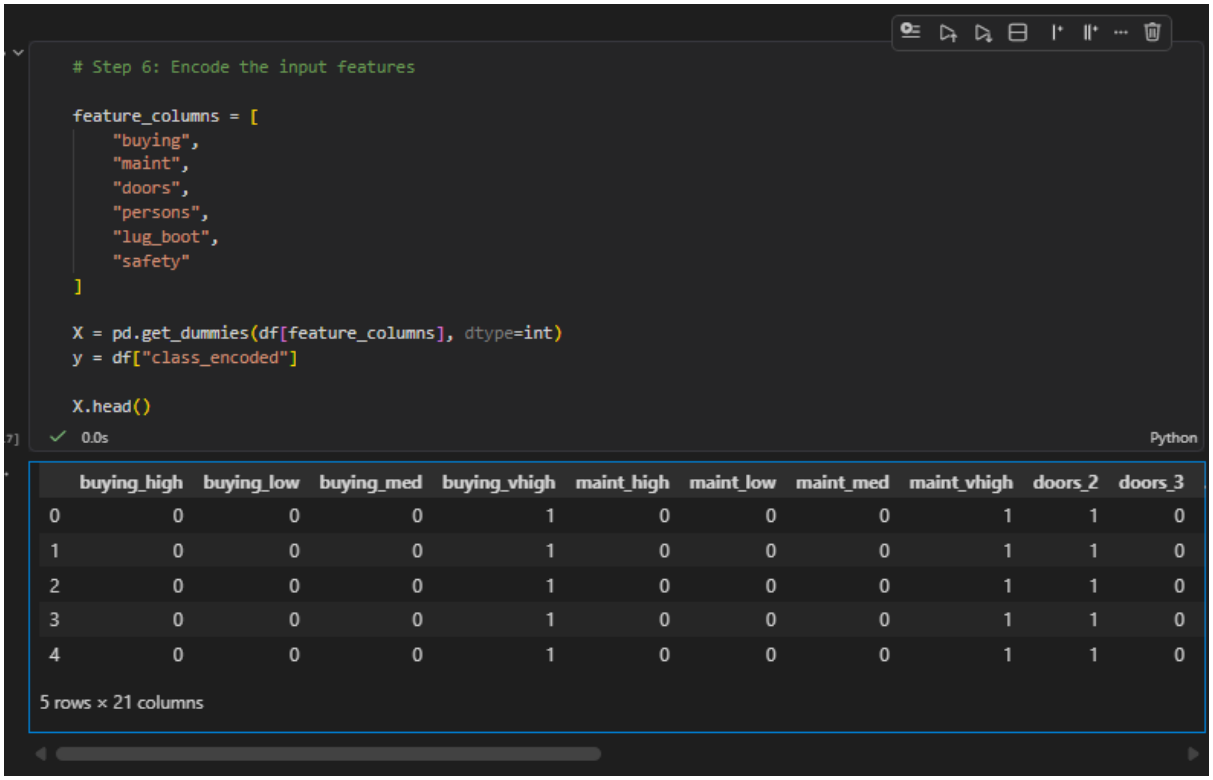
df[["class", "class_encoded"]].head(10)
```

0.3s

Python

	class	class_encoded
0	unacc	2
1	unacc	2
2	unacc	2
3	unacc	2
4	unacc	2
5	unacc	2
6	unacc	2
7	unacc	2
8	unacc	2
9	unacc	2

- **Figure 4:** Target variable encoding



```
# Step 6: Encode the input features

feature_columns = [
    "buying",
    "maint",
    "doors",
    "persons",
    "lug_boot",
    "safety"
]

X = pd.get_dummies(df[feature_columns], dtype=int)
y = df["class_encoded"]

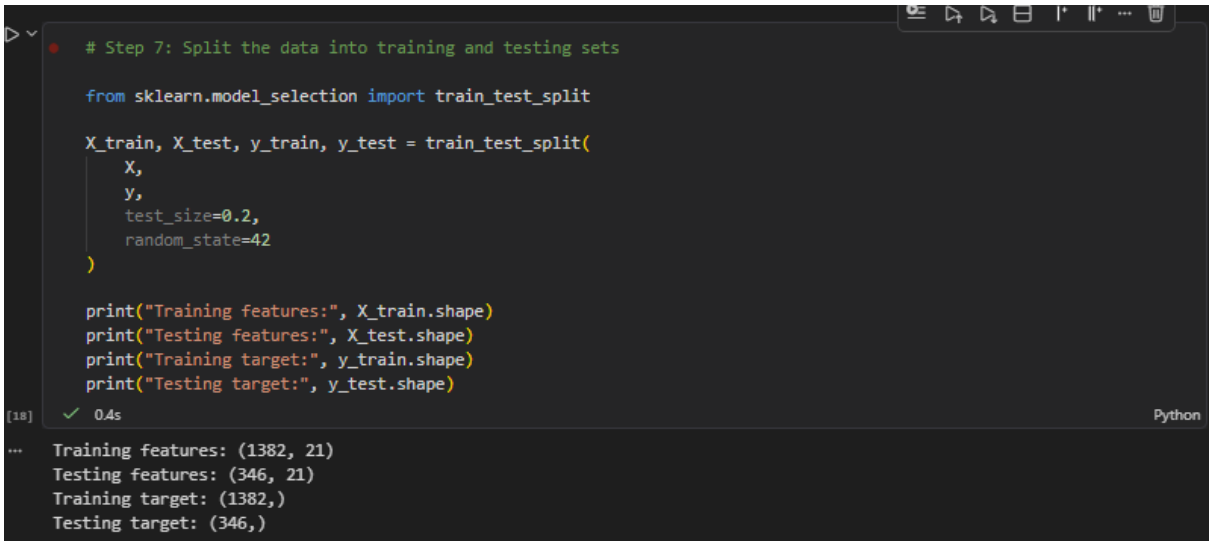
X.head()
```

0.0s Python

	buying_high	buying_low	buying_med	buying_vhigh	maint_high	maint_low	maint_med	maint_vhigh	doors_2	doors_3
0	0	0	0	1	0	0	0	1	1	0
1	0	0	0	1	0	0	0	1	1	0
2	0	0	0	1	0	0	0	1	1	0
3	0	0	0	1	0	0	0	1	1	0
4	0	0	0	1	0	0	0	1	1	0

5 rows × 21 columns

- **Figure 5:** One-hot encoded features



```
# Step 7: Split the data into training and testing sets

from sklearn.model_selection import train_test_split

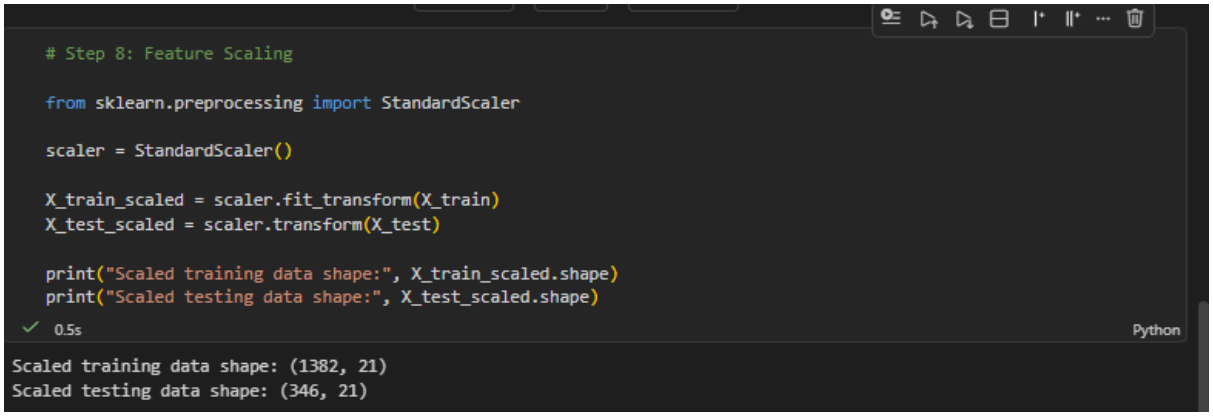
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

0.4s Python

```
Training features: (1382, 21)
Testing features: (346, 21)
Training target: (1382,)
Testing target: (346,)
```

- **Figure 6:** Training and testing split



- Figure 7: Feature scaling.

3. Model Implementation

3.1 K-Nearest Neighbors (KNN)

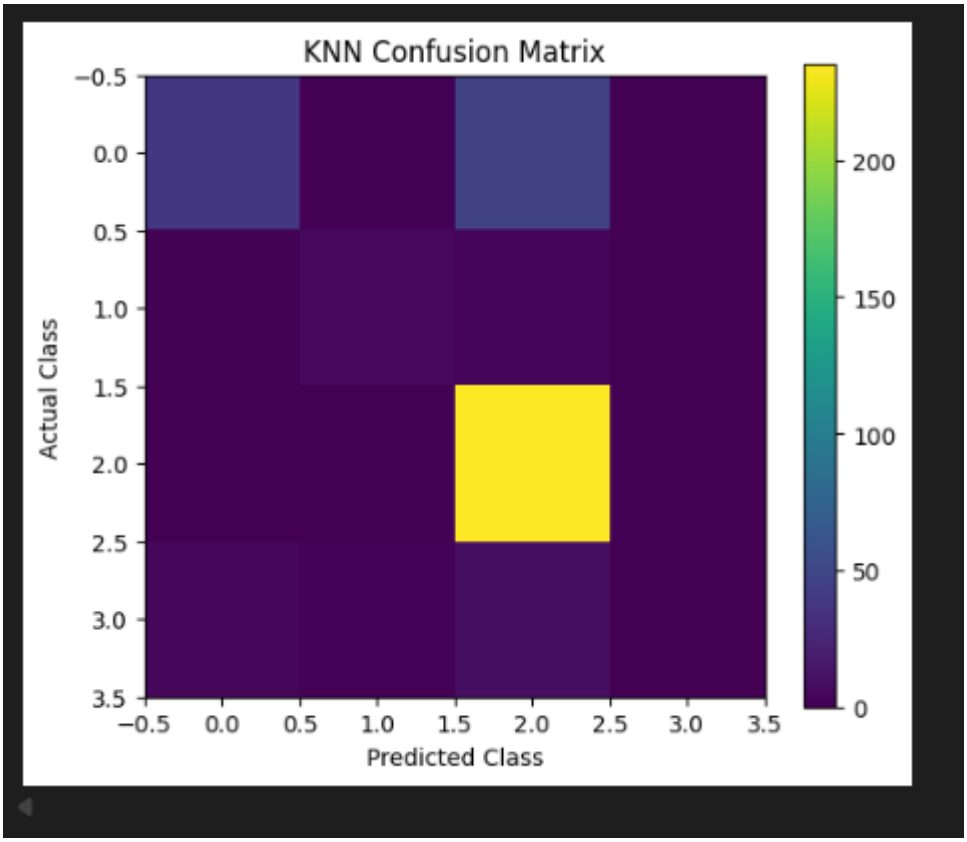
The first classification model implemented was the **K-Nearest Neighbors (KNN)** classifier. KNN classifies a new observation based on the classes of its nearest neighboring observations. Since KNN is distance-based, the standardized features produced during preprocessing were used.

The model was configured with **21 neighbors (k = 21)**. It was trained using the training dataset and then used to predict the classes of the 346 observations in the testing dataset.

The KNN model achieved an accuracy of **80.64%** and a weighted ROC-AUC of **0.9812**. The confusion matrix and classification report were also used to examine its performance across the individual classes.

KNN results:

Metric	Result
Number of neighbors	21
Accuracy	**80.64%**
ROC-AUC	**0.9812**



- **Figure 8:** KNN Confusion Matrix for k = 21

```python
# Step 12: KNN Classification Report (k=21)

from sklearn.metrics import classification_report

print("KNN Classification Report (k=21):")
print(classification_report(
    y_test,
    y_pred_knn,
    zero_division=0
))
```

[32] ✓ 0.0s Python

```
KNN Classification Report (k=21):
              precision    recall  f1-score   support

           0       0.86      0.46      0.60        83
           1       0.75      0.55      0.63        11
           2       0.80      1.00      0.89       235
           3       0.00      0.00      0.00        17

    accuracy                           0.81       346
   macro avg       0.60      0.50      0.53       346
weighted avg       0.77      0.81      0.77       346
```

- **Figure 9:** KNN Classification Report for k = 21

3.2 Logistic Regression

The second model implemented was **Logistic Regression**. Although the assignment refers to the regression component, Logistic Regression was selected because the Car Evaluation target variable is categorical with four classes. Logistic Regression is therefore appropriate for this classification problem.

The model was trained using the same training data and evaluated on the same testing data used for KNN. This ensured that the performance comparison between the two models was based on identical data.

Logistic Regression achieved an accuracy of **92.77%** and a weighted ROC-AUC of **0.9917**. Its classification report showed strong precision, recall, and F1-scores across the four classes.

Logistic Regression results:

Metric	Result
Accuracy	**92.77%**
ROC-AUC	**0.9917**

```python
# Step 14: Evaluate Logistic Regression Accuracy

from sklearn.metrics import accuracy_score, classification_report

log_accuracy = accuracy_score(y_test, y_pred_log)

print("Logistic Regression Accuracy:", round(log_accuracy * 100, 2), "%")

print("\nLogistic Regression Classification Report:")
print(classification_report(
    y_test,
    y_pred_log,
    zero_division=0
))
```

0.0s Python

```
Logistic Regression Accuracy: 92.77 %

Logistic Regression Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.81      0.85        83
           1       0.59      0.91      0.71        11
           2       0.96      0.97      0.97       235
           3       0.94      0.94      0.94        17

    accuracy                           0.93       346
   macro avg       0.85      0.91      0.87       346
weighted avg       0.93      0.93      0.93       346
```

- **Figure 10:** Logistic Regression Accuracy and Classification Report

```python
# Step 15: Logistic Regression Confusion Matrix

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

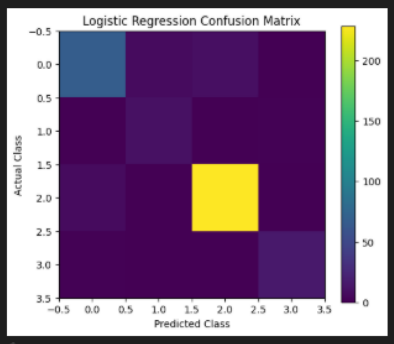
cm_log = confusion_matrix(y_test, y_pred_log)

print("Logistic Regression Confusion Matrix:")
print(cm_log)

plt.figure(figsize=(6, 5))
plt.imshow(cm_log)
plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.colorbar()
plt.show()
```

0.4s Python

```
Logistic Regression Confusion Matrix:
[[ 67   7   9   0]
 [  0  10   0   1]
 [  7   0 228   0]
 [  1   0   0  16]]
```



- **Figure 11:** Logistic Regression Confusion Matrix

```python
# Step 16: Logistic Regression ROC-AUC

from sklearn.metrics import roc_auc_score

y_proba_log = log_reg.predict_proba(X_test_scaled)

log_roc_auc = roc_auc_score(
    y_test,
    y_proba_log,
    multi_class="ovr",
    average="weighted"
)

print("Logistic Regression ROC-AUC:", round(log_roc_auc, 4))
```

0.6s Python

```
Logistic Regression ROC-AUC: 0.9917
```

- **Figure 12:** Logistic Regression ROC-AUC

3.3 Model Comparison

```python
# Step 18: Model Comparison Table

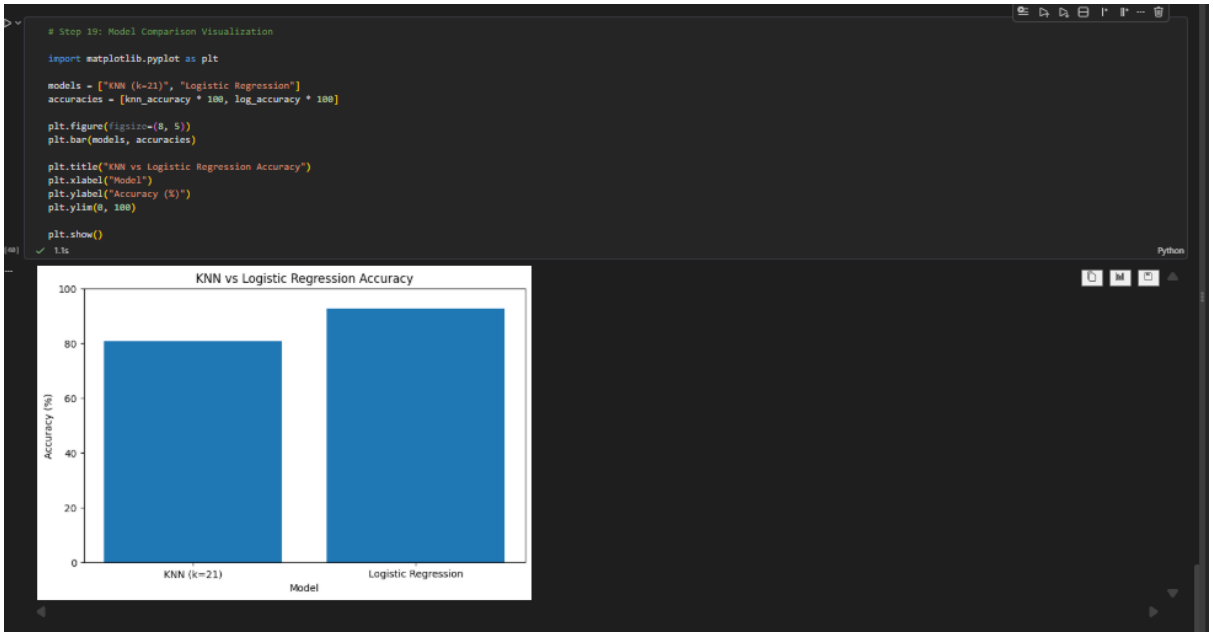
comparison = pd.DataFrame({
    "Model": ["KNN (k=21)", "Logistic Regression"],
    "Accuracy (%)": [
        round(knn_accuracy * 100, 2),
        round(log_accuracy * 100, 2)
    ],
    "ROC-AUC": [
        round(knn_roc_auc, 4),
        round(log_roc_auc, 4)
    ]
})

comparison
```

0.0s Python

	Model	Accuracy (%)	ROC-AUC
0	KNN (k=21)	80.64	0.9812
1	Logistic Regression	92.77	0.9917

- **Figure 13:** Model Performance Comparison Table



- **Figure 14:** Accuracy Comparison of KNN and Logistic Regression

4. Comparison and Discussion

Comparison of Models

The two models were evaluated using the same training and testing datasets. The results show that **Logistic Regression performed better than KNN (k = 21)** on both evaluation metrics.

KNN achieved an accuracy of **80.64%** and a ROC-AUC of **0.9812**, while Logistic Regression achieved an accuracy of **92.77%** and a ROC-AUC of **0.9917**.

Logistic Regression therefore improved the classification accuracy by **12.13 percentage points** compared with KNN. It also achieved a higher ROC-AUC, indicating stronger overall class discrimination on the test data.

The difference in performance may be partly explained by the structure of the encoded dataset. KNN makes predictions based on distances between observations. Although feature scaling was applied, one-hot encoded categorical features can still produce distance relationships that are not always ideal for nearest-neighbor classification.

Logistic Regression, on the other hand, learns relationships between the input features and the target classes during training. Its stronger performance on this dataset suggests that the learned decision boundaries were better suited to distinguishing the four car evaluation categories.

The confusion matrices also supports this observation. Logistic Regression correctly classified a large proportion of observations across all four classes, while KNN had considerably more difficulty with some of the less frequent classes.

Based on the accuracy and ROC-AUC results, **Logistic Regression is the better-performing model for this Car Evaluation classification task**.

Final comparison table

Model	Accuracy (%)	ROC-AUC
KNN (k = 21)	80.64	0.9812
Logistic Regression	**92.77**	**0.9917**

```python
# Step 18: Model Comparison Table

comparison = pd.DataFrame({
    "Model": ["KNN (k=21)", "Logistic Regression"],
    "Accuracy (%)": [
        round(knn_accuracy * 100, 2),
        round(log_accuracy * 100, 2)
    ],
    "ROC-AUC": [
        round(knn_roc_auc, 4),
        round(log_roc_auc, 4)
    ]
})

comparison
```

0.0s Python

	Model	Accuracy (%)	ROC-AUC
0	KNN (k=21)	80.64	0.9812
1	Logistic Regression	92.77	0.9917

Figure 13: Model Performance Comparison

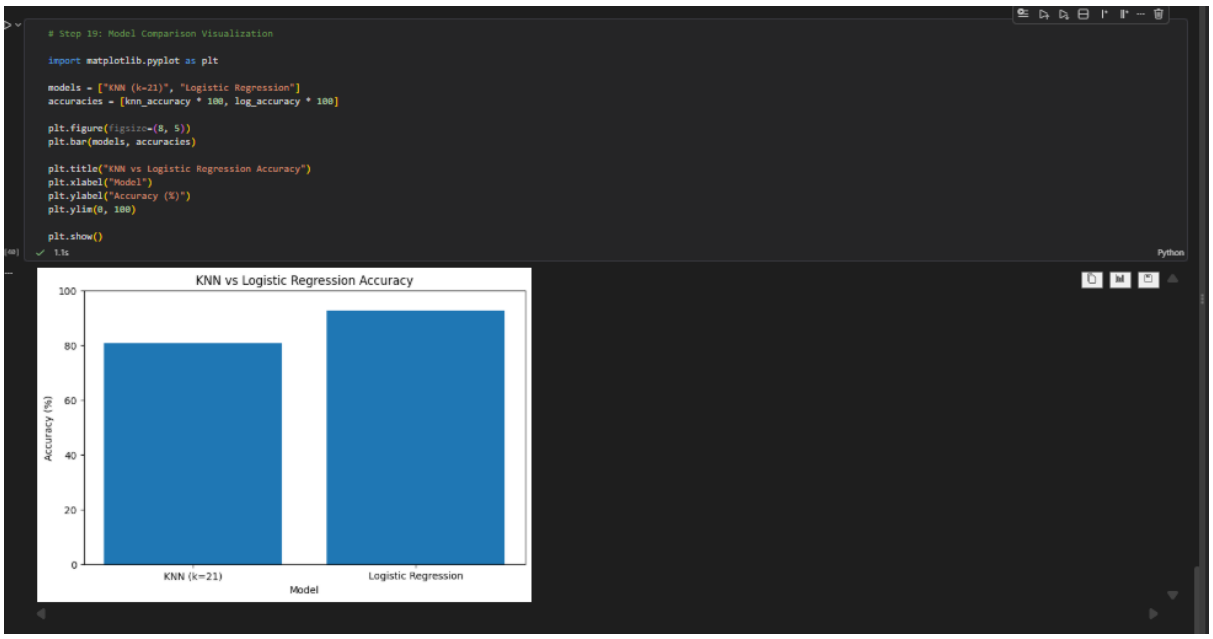


Figure 14: Accuracy Comparison of KNN and Logistic Regression

5. Conclusion

Conclusion

This practical successfully applied two supervised machine learning classification algorithms, K-Nearest Neighbors and Logistic Regression, to the UCI Car Evaluation dataset. The data was cleaned, categorical variables were encoded, the dataset was divided into training and testing sets, and feature scaling was applied before model training.

KNN with 21 neighbors achieved an accuracy of **80.64%** and a ROC-AUC of **0.9812**. Logistic Regression achieved a higher accuracy of **92.77%** and a ROC-AUC of **0.9917**.

Therefore, Logistic Regression performed better than KNN on the selected dataset and test split. The results demonstrate the importance of selecting a suitable machine learning algorithm based on the characteristics of the dataset and target variable.

Overall, the practical provided an effective comparison of two classification approaches and demonstrated the complete machine learning workflow from data preprocessing to model evaluation and comparison.

6. References

References

UCI Machine Learning Repository. (n.d.). *Car Evaluation Dataset*. https://archive.ics.uci.edu/dataset/19/car+evaluation

UCI Machine Learning Repository. (n.d.). *Car Evaluation Dataset — Raw Data*. https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data

Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. https://scikit-learn.org/

Final figure checklist

Figure	Content
1	UCI dataset/source
2	`df.head()` and `df.info()`
3	Missing-value check
4	Target encoding
5	One-hot encoding
6	Train/test split
7	Feature scaling
8	KNN confusion matrix
9	KNN classification report
10	Logistic Regression classification results
11	Logistic Regression confusion matrix
12	Logistic Regression ROC-AUC
13	Model comparison table
14	Accuracy comparison chart

Final results

KNN (k = 21)

- Accuracy: **80.64%**
- ROC-AUC: **0.9812**

Logistic Regression

- Accuracy: **92.77%**
- ROC-AUC: **0.9917**

Best model: Logistic Regression